In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom
from scipy.optimize import brentq
```

I ACCEPT THE HONOUR CODE

Q1 — Auditing Differential Privacy

Empirical privacy auditing via the Steinke et al. algorithm: run a membership inference attack using canary datapoints, then extract a high-probability lower bound on ε.

In [ ]:
```
def sigma_from_eps(eps, delta=1e-6):
    return np.sqrt(2 * np.log(1.25 / delta)) / eps


def compute_eps_hat(n_correct, n_total):
    def f(eps):
        p = np.exp(eps) / (1 + np.exp(eps))
        return binom.sf(n_correct - 1, n_total, p) - 0.05
    if f(0) >= 0:
        return 0.0
    return brentq(f, 0, 50)


def run_audit(d, m, sigma, rng):
    u = rng.standard_normal((m, d)).astype(np.float32)
    canaries = u / np.linalg.norm(u, axis=1, keepdims=True)

    S = rng.choice([-1, 1], size=m)

    theta_det = canaries[S == 1].sum(axis=0)
    noise = rng.standard_normal(d).astype(np.float32) * sigma
    theta = theta_det + noise

    scores = canaries @ theta
    order = np.argsort(scores)

    best = 0.0
    for m1 in range(1, min(m // 2, 500) + 1):
        T = np.zeros(m, dtype=int)
        T[order[-m1:]] = 1
        T[order[:m1]]  = -1
        n_correct = int(np.sum((T != 0) & (S == T)))
        n_total   = 2 * m1
        val = compute_eps_hat(n_correct, n_total)
        if val > best:
            best = val
    return best
```

Q 1.1 — Vary ε ($d = 10^4$, $m = 1000$)

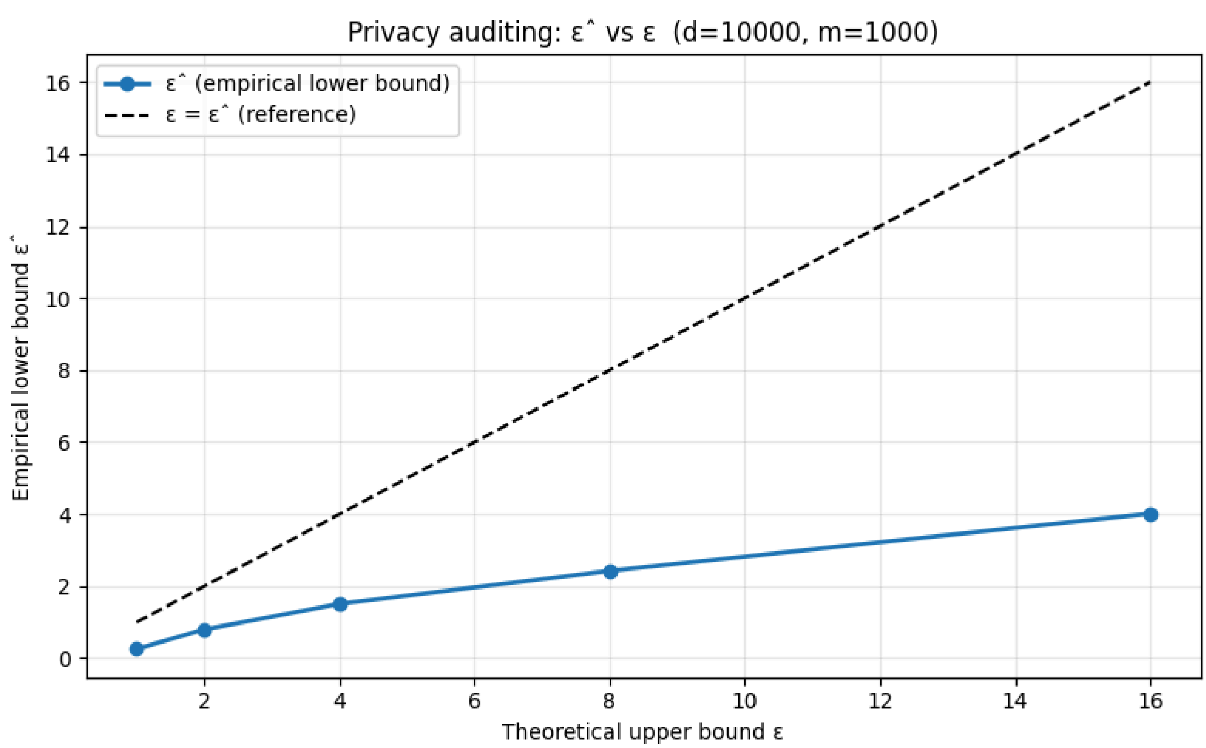
In [3]:
```python
rng = np.random.default_rng(42)

d, m = 10_000, 1000
eps_vals = [1, 2, 4, 8, 16]

eps_hats_q1 = [run_audit(d, m, sigma_from_eps(eps), rng) for eps in eps_vals

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(eps_vals, eps_hats_q1, marker='o', linewidth=2, label='εˆ (empirical
ax.plot(eps_vals, eps_vals, 'k--', linewidth=1.5, label='ε = εˆ (reference)'
ax.set_xlabel('Theoretical upper bound ε')
ax.set_ylabel('Empirical lower bound εˆ')
ax.set_title(f'Privacy auditing: εˆ vs ε  (d={d}, m={m})')
ax.legend()

ax.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.savefig('q1_1.png', dpi=150)
plt.show()
```



Trend (Q 1.1): εˆ increases as the theoretical ε grows. At small ε (high σ, lots of noise), the adversary's score barely separates members from non-members, so the empirical bound is near zero. At larger ε the noise σ = sqrt(2 log(1.25/δ))/ε shrinks, the signal x_i^T θ is more concentrated around 1 for members and 0 for non-members, and the attack succeeds more reliably, pushing εˆ up. The empirical bound stays below the theoretical ε, as expected (it is a lower bound), but tracks it closely at large ε where the auditing has enough power.

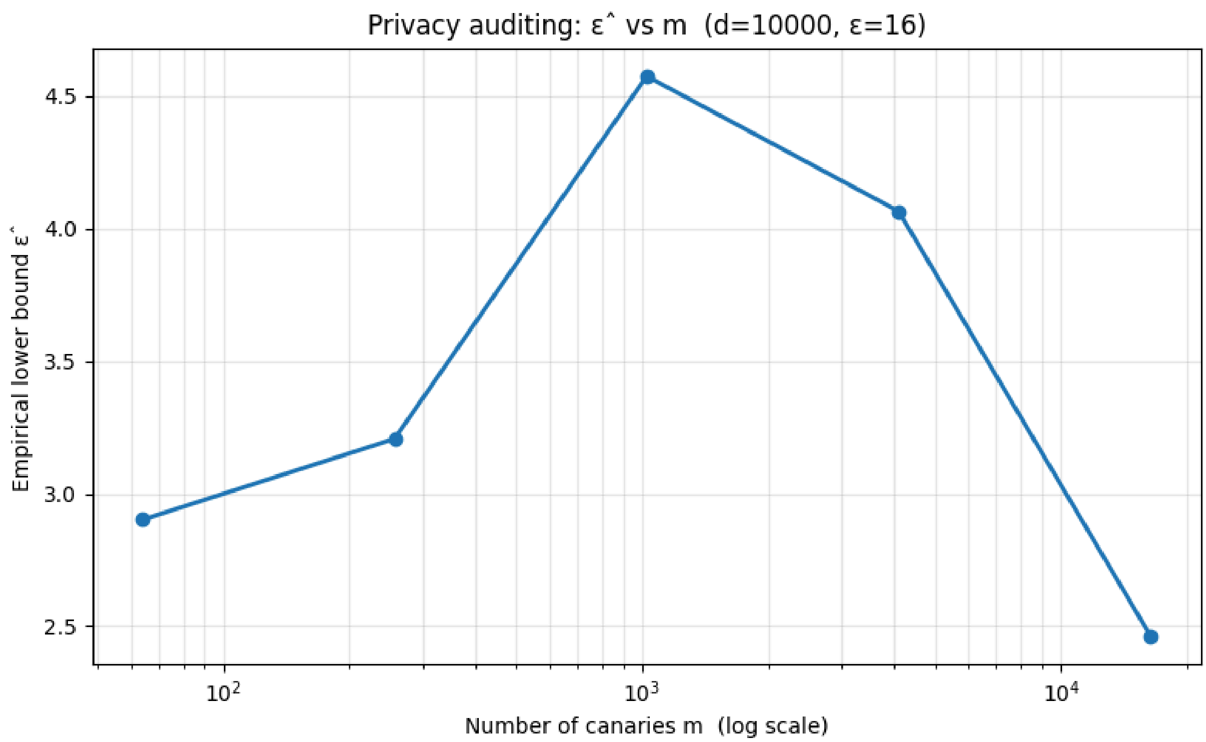
Q 1.2 — Vary m (d = 10^4, ε = 16)

In [4]:
```
rng = np.random.default_rng(42)

d     = 10_000
sig   = sigma_from_eps(16)
m_vals = [2**6, 2**8, 2**10, 2**12, 2**14]

eps_hats_q2 = [run_audit(d, mv, sig, rng) for mv in m_vals]

fig, ax = plt.subplots(figsize=(8, 5))
ax.semilogx(m_vals, eps_hats_q2, marker='o', linewidth=2)
ax.set_xlabel('Number of canaries m  (log scale)')
ax.set_ylabel('Empirical lower bound ε̂')
ax.set_title(f'Privacy auditing: ε̂ vs m  (d={d}, ε=16)')
ax.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.savefig('q1_2.png', dpi=150)
plt.show()
```



Trend (Q 1.2): ε̂ rises as m grows from small values, then levels off or even dips at very large m. With few canaries the estimator has high variance (only a handful of guesses, so N_correct is noisy), giving a weak bound. As m grows, the law of large numbers kicks in and the attack stabilises. However, adding more canaries also increases interference: the Gaussian mechanism output θ = Σ_{S_i=+1} x_i + noise is contaminated by the sum of O(m/2) random unit vectors, which introduces an O(sqrt(m)) term in the noise. For sufficiently large m this cross-canary "signal pollution" overwhelms the single-canary signal of 1, degrading the bound. The sweet spot is the bias-variance tradeoff: moderate m balances statistical power against this interference.

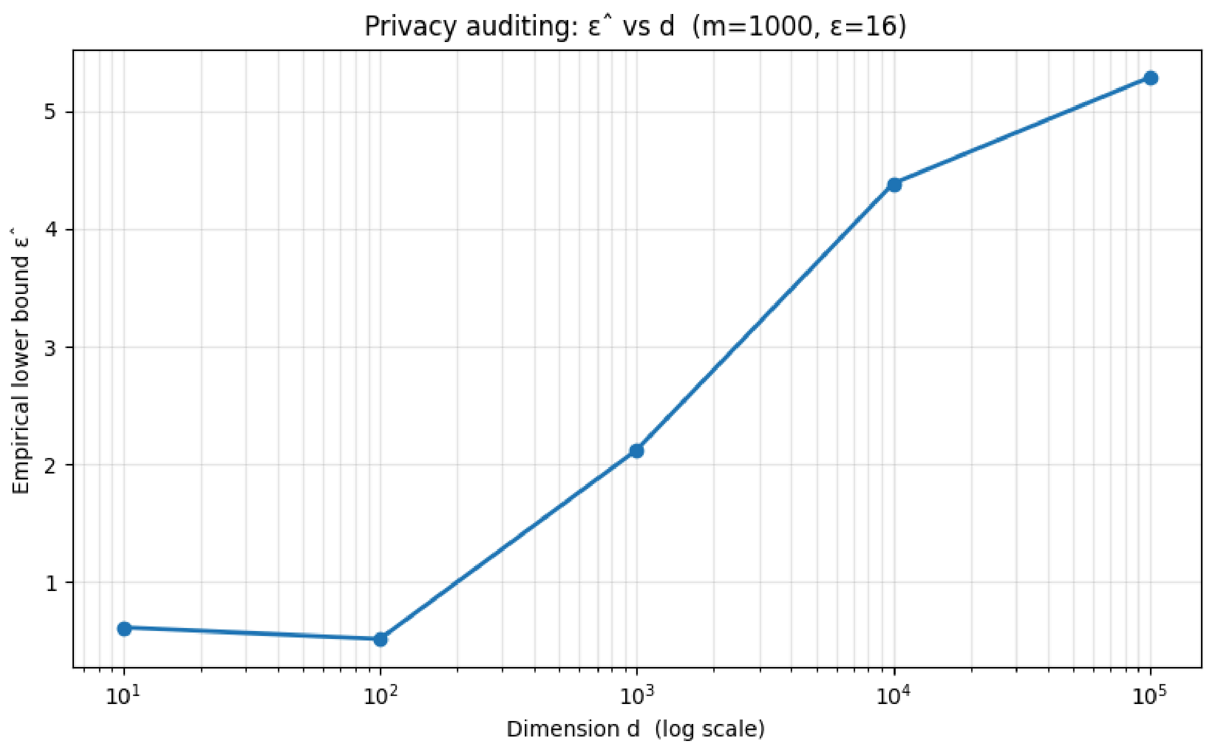
Q 1.3 — Vary d (m = 1000, ε = 16)

In [5]:
```
rng = np.random.default_rng(42)

m   = 1000
sig = sigma_from_eps(16)
d_vals = [10, 100, 1_000, 10_000, 100_000]

eps_hats_q3 = [run_audit(dv, m, sig, rng) for dv in d_vals]

fig, ax = plt.subplots(figsize=(8, 5))
ax.semilogx(d_vals, eps_hats_q3, marker='o', linewidth=2)
ax.set_xlabel('Dimension d  (log scale)')
ax.set_ylabel('Empirical lower bound ε̂')
ax.set_title(f'Privacy auditing: ε̂ vs d  (m={m}, ε=16)')
ax.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.savefig('q1_3.png', dpi=150)
plt.show()
```



Trend (Q 1.3): ε̂ increases monotonically with d. The key insight is concentration of measure: in high dimensions, two random unit vectors x_i, x_j satisfy x_i^T x_j ≈ 0 with variance O(1/d). At low d, the inner products between distinct canaries are non-negligible, so the scores of non-members are contaminated by the projection onto other canaries that happen to be in D+. At high d, canaries are nearly orthogonal, so E[score | S_i = +1] ≈ 1 while E[score | S_i = -1] ≈ 0 with very small variance from cross-terms. This makes the membership inference task progressively easier, and the empirical bound approaches the theoretical ε=16.